	The complete list is shown in appendix A.4.	modularity.	Suspense.
`use` API	Learn the signature and purpose of the `use()` for reading resources during rendering (e.g., context or async data). See appendix A.4 for details.	Use `use(resource)` to access data or context in Server Components or Concurrent features. Integrate with async workflows effectively.	Architect resource-heavy or data-fetching components using `use()` alongside Suspense, error boundaries, and streaming for scalability.
Custom Hooks	Encapsulate shared logic into reusable hooks. See appendix A.4 for best practices.	Build modular hooks to avoid duplication. Compose multiple hooks.	Architect hook-based libraries and dynamic hook factories.

Having explored React's builtin and custom hooks, let's move on to tackle more interview questions.

Initial question (applied skill): "Why is the `useFormStatus` hook imported from `react-dom` instead of `react`, and what is its primary use case?"

Analysis: This question assesses a candidate's understanding of React's architecture and the distinction between core React functionality and DOM-specific features. It evaluates knowledge of React Server Components, progressive enhancement, and how React separates concerns between the core library (`react`) and the rendering environment (`react-dom`).

Answer: The `useFormStatus` hook (https://react.dev/reference/react-dom/hooks/useFormStatus) is imported from `react-dom` rather than `react` because it is closely tied to the rendering environment and browser-specific behaviors related to forms. Unlike core React hooks that manage general